

HELIXCODE_QUICK_REFERENCE

HelixCode Quick Reference Guide

Provider File Locations

Provider	File Path	API Endpoint	Context Window
Anthropic Claude	/internal/llm/anthropic_provider.go	https://api.anthropic.com/v1	200K tokens
Google Gemini	/internal/llm/gemini_provider.go	https://generativelanguage.googleapis.com	2M tokens
OpenAI	/internal/llm/openai_provider.go	https://api.openai.com/v1	1M+ tokens
Qwen	/internal/llm/qwen_provider.go	https://dashscope.aliyuncs.com	Variable
xAI (Grok)	/internal/llm/xai_provider.go	https://api.x.ai/v1	Variable
OpenRouter	/internal/llm/openrouter_provider.go	https://openrouter.ai/api/v1	Variable
GitHub Copilot	/internal/llm/copilot_provider.go	https://api.githubcopilot.com	Variable
Local (Llama.cpp)	/internal/llm/local_provider.go	http://localhost:11434	Variable
Ollama	/internal/llm/ollama_provider.go	http://localhost:11434	Variable
Llama.cpp	/internal/llm/llamacpp_provider.go	Configurable	Variable

Core Features Location Map

Feature	File Path	Key Type/Function
Provider Interface	/internal/llm/provider.go	type Provider interface (line 112)
Provider Manager	/internal/llm/provider.go	type ProviderManager struct (line 151)

Feature	File Path	Key Type/Function
Model Manager	/internal/llm/ model_manager.go	type ModelManager struct (line 16)
Tool Calling	/internal/llm/ tool_provider.go	type EnhancedLLMProvider interface (line 46)
Reasoning Engine	/internal/llm/ reasoning.go	type ReasoningEngine struct (line 73)
MCP Protocol	/internal/mcp/ server.go	type MCPServer struct (line 17)
Task Manager	/internal/task/ manager.go	type TaskManager struct (line 98)
Worker Pool	/internal/ worker/ ssh_pool.go	type SSHWorkerPool struct (line 17)
Workflow Engine	/internal/ workflow/ executor.go	Workflow execution logic
Configuration	/config/ config.yaml	YAML configuration file

Environment Variables

Essential

HELIX_AUTH_JWT_SECRET=your-jwt-secret
HELIX_DATABASE_PASSWORD=your-db-password

API Keys

ANTHROPIC_API_KEY=sk-ant-xxx
OPENAI_API_KEY=sk-xxx
GEMINI_API_KEY=your-key
GITHUB_TOKEN=ghp_xxx
QWEN_API_KEY=your-key
XAI_API_KEY=xai-xxx
OPENROUTER_API_KEY=sk-or-xxx

Optional

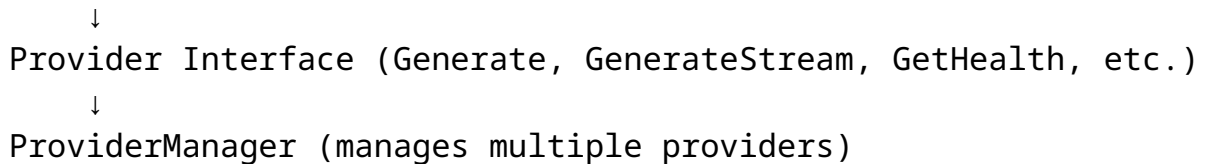
```
HELIX_REDIS_PASSWORD=your-redis-password
HELIX_SLACK_WEBHOOK_URL=https://hooks.slack.com/...
HELIX_TELEGRAM_BOT_TOKEN=xxx
HELIX_TELEGRAM_CHAT_ID=xxx
HELIX_DISCORD_WEBHOOK_URL=https://discord.com/api/...
```

Key Architectural Patterns

1. Provider Pattern

All LLM providers implement the Provider interface at `provider.go:112`

Concrete Provider (e.g., `AnthropicProvider`)



2. Factory Pattern

Create providers dynamically with `ProviderFactory.CreateProvider()` at `provider.go:339`

3. Model Selection Pattern

`ModelManager.SelectOptimalModel()` at `model_manager.go:75` scores models based on: - Required capabilities - Context window - Hardware compatibility - Task type suitability - Quality preferences

4. Task Distribution Pattern

Tasks flow through: Queue → Priority Selection → Worker Assignment → Execution → Checkpoint

5. MCP Protocol Pattern

WebSocket → Session Management → Tool Execution → Broadcasting

Advanced Features Matrix

Feature	Provider	Implementation	Status
Extended Thinking	Anthropic	anthropic_provider.go:86	Complete
Prompt Caching	Anthropic	anthropic_provider.go:48	Complete
Vision Support	Claude, Gemini, OpenAI	anthropicImageSource	Complete
Streaming	All	GenerateStream() in provider.go:122	Complete
Function Calling	All	tool_provider.go:46	Complete
Reasoning	All	reasoning.go:73	Complete
MCP Protocol	Server-side	mcp/server.go:17	Complete
Task Checkpointing	Core	task/checkpoint.go	Complete
Worker Auto-Install	SSH Workers	ssh_pool.go:77	Complete

Testing Files

Test Category	Files
Unit Tests	*_test.go alongside source files
Integration Tests	/test/integration/
E2E Tests	/test/e2e/
Automation Tests	/test/automation/
Load Tests	/test/load/

Provider Tests

- anthropic_provider_test.go
- gemini_provider_test.go
- qwen_provider_test.go
- reasoning_test.go
- integration_test.go

Build Commands Quick Reference

Development

<code>make build</code>	<code># Single binary</code>
<code>make dev</code>	<code># Build and run with dev config</code>
<code>make test</code>	<code># Run all tests</code>
<code>make fmt</code>	<code># Format code</code>
<code>make lint</code>	<code># Lint code</code>

Production

<code>make prod</code>	<code># Cross-platform binaries</code>
<code>make clean</code>	<code># Clean artifacts</code>

Platform-specific

<code>make mobile-ios</code>	<code># Build iOS framework</code>
<code>make mobile-android</code>	<code># Build Android AAR</code>
<code>make aurora-os</code>	<code># Aurora OS client</code>
<code>make symphony-os</code>	<code># Symphony OS client</code>

Utilities

<code>make logo-assets</code>	<code># Generate logo assets</code>
-------------------------------	-------------------------------------

API Endpoints

POST	<code>/api/auth/register</code>	- Register user
POST	<code>/api/auth/login</code>	- Login
POST	<code>/api/auth/logout</code>	- Logout
GET	<code>/api/auth/me</code>	- Current user
POST	<code>/api/auth/refresh</code>	- Refresh token
GET	<code>/api/tasks</code>	- List tasks
POST	<code>/api/tasks</code>	- Create task
GET	<code>/api/tasks/{id}</code>	- Get task details
PUT	<code>/api/tasks/{id}</code>	- Update task
DELETE	<code>/api/tasks/{id}</code>	- Delete task
GET	<code>/api/workers</code>	- List workers
POST	<code>/api/workers</code>	- Register worker
GET	<code>/api/workers/{id}</code>	- Worker details
DELETE	<code>/api/workers/{id}</code>	- Remove worker
GET	<code>/health</code>	- Health check
WS	<code>/ws/mcp</code>	- MCP protocol (WebSocket)

Important Type Definitions

Request/Response Types

- `LLMRequest` (provider.go:44): Unified LLM request format
- `LLMResponse` (provider.go:79): Unified LLM response format
- `Tool` (provider.go:66): Function definition
- `ToolCall` (provider.go:92): Function call execution
- `Usage` (provider.go:105): Token usage tracking

Provider-Specific Types

- `anthropicRequest`, `anthropicResponse` (anthropic_provider.go)
- `geminiRequest`, `geminiResponse` (gemini_provider.go)
- `openaiRequest` (openai_provider.go)
- Custom types for each provider API

Task Types

- `Task` (task/manager.go:74): Distributed task
- `TaskType`: planning, building, testing, refactoring, debugging, design, diagram, deployment, porting
- `TaskPriority`: Low(1), Normal(5), High(10), Critical(20)
- `TaskStatus`: pending, assigned, running, completed, failed, paused

Workflow Types

- `Workflow` (workflow/workflow.go:7): Workflow definition
- `Step` (workflow/workflow.go:19): Workflow step
- `StepType`: analysis, generation, execution, validation
- `StepAction`: analyze_code, generate_code, run_tests, lint_code, build_project

Worker Types

- `SSHWorker` (worker/ssh_pool.go:24): SSH-accessible worker
- `SSHWorkerPool` (worker/ssh_pool.go:17): Worker pool manager
- `WorkerConfig` (worker/types.go:12): Worker configuration

Model Capabilities

Available for `SelectOptimalModel()` criteria: - `CapabilityTextGeneration`: Text output - `CapabilityCodeGeneration`: Code synthesis - `CapabilityCodeAnalysis`: Code review - `CapabilityPlanning`: Requirement

analysis - CapabilityDebugging: Error diagnosis - CapabilityRefactoring:
Code optimization - CapabilityTesting: Test generation - CapabilityVision:
Image understanding

Common Configuration Values

```
# Server defaults
address: "0.0.0.0"
port: 8080
read_timeout: 30
write_timeout: 30

# Database
host: "localhost"
port: 5432
dbname: "helixcode"

# Workers
health_check_interval: 30
max_concurrent_tasks: 10

# Tasks
max_retries: 3
checkpoint_interval: 300

# LLM
max_tokens: 4096
temperature: 0.7
```

Debugging Tips

1. **Check provider availability:** `provider.IsAvailable(ctx)`
2. **Monitor health:** `ProviderManager.GetProviderHealth(ctx)`
3. **View active sessions:** `MCPServer.GetSessionCount()`
4. **Track tasks:** Query task status from `TaskManager`
5. **Worker status:** Check `SSHWorkerPool` health checks
6. **Token usage:** Monitor `Usage` struct in responses

Performance Considerations

1. **Prompt Caching:** Enabled by default for Anthropic (70-90% savings)
2. **Streaming:** Use `GenerateStream()` for real-time responses
3. **Model Selection:** Auto-selects optimal model for task type

4. **Worker Pool:** SSH connections reused for efficiency
5. **Task Checkpointing:** Automatic every 300 seconds

Security Notes

- JWT tokens expire after 86400 seconds (24 hours)
- API keys stored in environment variables, never in config
- SSH connections use key-based authentication
- MCP tool execution respects permission lists
- Database passwords via env vars (HELIX_DATABASE_PASSWORD)